楼层平面图：DXF 自动抽取 + 手绘风格渲染 混合方案

> 版本：v1.0
>
> 日期：2026-04-29
>
> 作者：PropOS 架构组
>
> 关联文档：
>
> - SVG_HOTZONE_SPEC.md — 现有 SVG 矢量底图 + 热区覆盖规范
>
> - HOTZONE_ANNOTATE_PLAN.md — 单元热区标注流水线
>
> - CAD_CONVERSION_SOP.md — DWG→DXF→SVG 转换标准流程
>
> - 原型参考：frontend/src/app/pages/FloorPlan.tsx

---

1. 背景与动机

1.1 现状（已实现）

后端流水线 scripts/split_dxf_by_floor.py 已能把 A 座 DXF 按楼层切分为：

```
floors/A座_F<N>.svg      # 矢量 CAD 线稿（中性化 currentColor）
floors/A座_F<N>.json     # floor_map 骨架（viewport + dxf_region + units 占位）
```

前端（Flutter / Admin / uni-app）按 SVG_HOTZONE_SPEC.md 加载：

- `<g id="floor-plan">` — 直接渲染 CAD 矢量（保留所有真实建筑细节）
- `<g id="unit-hotspots">` — 由 scripts/annotate_hotzone.py 标注后填充

优点：100% 还原真实 CAD，单元热区可点击。

缺点：CAD 视觉杂乱，含大量与租赁运营无关的细节（设备配筋、施工说明、节点大样、户型分析），对运营人员认知负担高。

1.2 期望状态（本方案）

参考前端原型 FloorPlan.tsx 的"语义化手绘"风格：

- 只保留运营关心的元素：**外轮廓 / 核心筒 / 电梯 / 楼梯 / 卫生间 / 柱网 / 单元**
- 像素级可控的视觉规范（圆角、阴影、状态色块、装修斜纹、北向标）
- 数据驱动，可在 Web/Mobile 各端用同一份 `floor_map.json` 渲染

1.3 决策：不走"完全自动生成 FloorPlan.tsx 风格 SVG"

完全自动从 DXF 反向语义化（识别"这是核心筒"/"这是电梯井"/"这是窗洞"）受限于：

1. 不同楼栋图层命名差异巨大（A 座 `WALL` / B 座可能 `0muqiang`）
2. CAD 浮点坐标 → "漂亮整数 viewBox" 必然失真

3. 手绘组件（电梯 ×、北向标、楼梯斜纹）画法千差万别

故采用「**两条渲染管线并存 + 数据扩展 + 半自动标注**」混合方案。

---

2. 架构总览

```
                    ┌──────────────────────────────────┐
                    │     scripts/split_dxf_by_floor.py    │
                    │      （扩展版：抽取结构化语义）      │
                    └──────────────────────────────────┘
                                  │
                  ┌───────────────┼────────────────┐
                  ▼               ▼                ▼
   A座_F<N>.svg            A座_F<N>.json        A座_F<N>.candidates.json
（CAD 矢量底图）      （扩展字段：        （结构候选项，
                         outline/structures/      待人工确认）
                         windows/north/units）
                  ▼               ▼                ▼
  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐
  │ 渲染模式 A        │  │ 渲染模式 B        │  │ 标注工具          │
  │ "矢量底图"        │  │ "语义手绘"        │  │ annotate_floor   │
  │ <g floor-plan>   │  │ 数据驱动重建      │  │ _structures.py   │
  │ + <g hotspots>   │  │ FloorPlan.tsx 风格 │  │（人工审核候选）   │
  └──────────────────┘  └──────────────────┘  └──────────────────┘
          ▲                     ▲
          │                     │
          └──────── 同一份 floor_map.json + 单元热区状态 ────────┘
```

2.1 两种渲染模式由 `floor_map.json.render_mode` 控制

值	含义	数据要求
`vector`（默认）	加载 SVG 矢量底图 + 单元热区覆盖	仅需 `units[]`
`semantic`	数据驱动重建 FloorPlan.tsx 风格	需完整 `outline` / `structures[]` / `windows[]` / `north` / `units[]`

前端按 `render_mode` 字段分支：

- Flutter：`flutter_svg` 加载 svg vs `CustomPaint` 数据驱动绘制
- Admin / uni-app：`<img src=svg>` vs Vue 组件循环渲染 `<rect>`

---

3. `floor_map.json` 扩展字段规范（v2）

3.1 完整结构

```
{
  "schema_version": "2.0",
  "render_mode": "vector",            // "vector" | "semantic"
  "floor_id": "uuid",                  // 后端绑定时填充
  "building_id": "A座",
  "floor_label": "F11",
  "svg_version": null,                 // 后端上传时填充
  "viewport": { "width": 1200, "height": 800 },

  // —— 矢量模式所需（已实现）——
  "dxf_region": {
    "min_x": 0.0, "min_y": 0.0,
    "max_x": 84100.0, "max_y": 59400.0
  },

  // —— 语义模式新增字段——
  "outline": {
    "type": "rect",                    // "rect" | "polygon"
    "x": 0, "y": 0, "w": 1200, "h": 800
    // type=polygon 时改用 "points": [[x,y], ...]
  },
  "structures": [
    {
      "type": "core",                  // 见 §3.2 类型枚举
      "rect": { "x": 470, "y": 230, "w": 260, "h": 340 },
      "label": "核心筒",
      "source": "auto"                 // "auto" | "manual"
    },
    {
      "type": "elevator",
      "rect": { "x": 485, "y": 245, "w": 50, "h": 75 },
      "code": "E1",
      "source": "auto"
    },
    { "type": "stair",     "rect": {...}, "label": "消防楼梯", "source": "auto" },
    { "type": "restroom",  "rect": {...}, "gender": "M",       "source": "manual" },
    { "type": "restroom",  "rect": {...}, "gender": "F",       "source": "manual" },
    { "type": "equipment", "rect": {...}, "label": "设备间",    "source": "manual" },
    { "type": "column",    "point": [0, 0],                    "source": "auto" },
    { "type": "corridor",  "rect": {...},                      "source": "manual" }
  ],
  "windows": [
    { "side": "N", "x": 30, "width": 30 },
    { "side": "S", "x": 30, "width": 30 },
    { "side": "E", "y": 120, "width": 20 },
    { "side": "W", "y": 120, "width": 20 }
  ],
```

```
  "north": { "x": 1140, "y": 60, "rotation_deg": 0 },

  "units": [
    {
      "unit_id": "uuid",
      "label": "A1101",
      "polygon": [[x1,y1],[x2,y2],...],
      "centroid": [x, y],
      "area_sqm": 120.5,
      "property_type": "office"
    }
  ]
}
```

3.2 `structures[].type` 枚举

type	中文	自动识别策略	渲染样式（参考 FloorPlan.tsx）
`core`	核心筒	包含电梯/楼梯/卫生间的最小外接矩形	灰色填充 + 标签"核心筒"
`elevator`	电梯	INSERT 块名含 "DT/电梯/ELEV" 或 layer="电梯"	矩形 + 对角线 × + 编号 E1/E2
`stair`	楼梯	INSERT 块名含 "LT/楼梯/STAIR" 或 HATCH 在该图层	斜纹 fill + 标签"消防楼梯"
`restroom`	卫生间	INSERT 块名含 "WC/卫生间/TOILET"	矩形 + "男卫"/"女卫"
`equipment`	设备间	layer 含 "设备/MEP"	矩形 + 标签
`corridor`	走廊	走廊 layer="PUB"（或人工）	浅灰背景条
`column`	柱	layer="柱"/HATCH 中心点	实心方块（约 10×10）

3.3 `source` 字段语义

- `"auto"` ：脚本自动抽取，未经人工审核
- `"manual"` ：人工在标注工具中确认或新增

后端在合并新版 DXF 时：

- `auto` 项可被新一轮抽取覆盖
- `manual` 项保留，除非人工显式删除

---

4. 后端实现：`split_dxf_by_floor.py` 扩展

4.1 新增模块拆分

为避免 `split_dxf_by_floor.py` 进一步膨胀，按职责拆分：

```
scripts/
├── split_dxf_by_floor.py            # 主入口（保持不变，调用下方模块）
├── floor_map/
│   ├── __init__.py
│   ├── outline_extractor.py          # 提取外轮廓
│   ├── structure_detector.py         # 识别 core/elevator/stair/restroom/equipment
│   ├── column_detector.py            # 识别柱位
│   ├── window_detector.py            # 识别窗洞（基于墙体断点 / WINDOW 图层）
│   └── coordinate_mapper.py          # DXF 坐标 → SVG viewport 坐标
└── annotate_floor_structures.py      # 半自动标注工具（CLI / Web）
```

4.2 抽取流程

```
# scripts/split_dxf_by_floor.py 在每个楼层渲染后追加:

from floor_map import (
    outline_extractor,
    structure_detector,
    column_detector,
    window_detector,
    coordinate_mapper,
)

# 1. 收集楼层区域内的 DXF 实体
entities = collect_entities_in_region(msp, region)

# 2. DXF mm 坐标 → SVG viewport 坐标
mapper = coordinate_mapper.build(actual_bb, viewport=(1200, 800))

# 3. 自动抽取
outline    = outline_extractor.extract(entities, mapper)
structures = structure_detector.detect(entities, mapper, doc.blocks)
structures += column_detector.detect(entities, mapper)
windows    = window_detector.detect(entities, mapper)

# 4. 写入 .json 和 .candidates.json
floor_map["outline"]    = outline
floor_map["structures"] = structures
floor_map["windows"]    = windows
floor_map["render_mode"] = "vector"  # 默认仍是 vector，等人工切换
```

4.3 各探测器约束

模块	必须返回	不允许做的事
`outline_extractor`	`{type, x, y, w, h}` 或 polygon 点集	不得修改 DXF
`structure_detector`	`List[Structure]`，`source="auto"`	不识别窗洞、柱、单元
`column_detector`	`List[Column]`，每个 `{type:"column", point:[x,y]}`	不归并到 structure
`window_detector`	`List[Window]`，按 N/S/E/W 分组	输出 ≤ 100 个/层（超出时按等距聚类）
`coordinate_mapper`	`dxf_to_viewport(x, y) → (sx, sy)`	必须保持长宽比；y 翻转（DXF 上正、SVG 下正）

4.4 命名规则

- 单元 label：沿用 `annotate_hotzone.py` 已有逻辑（基于楼层号 + 房间号）
- 电梯编号：按 X 坐标从左到右 `E1, E2, E3, ...`
- 卫生间 gender：根据邻近 TEXT "男卫"/"女卫" 判断；歧义时 `source="auto"` 标 `gender:"unknown"` 待人工确认

---

5. 半自动标注工具：`annotate_floor_structures.py`

5.1 功能

读取 `*.candidates.json` + `*.svg`，提供：

- 列出所有 `source="auto"` 项，运营/项目工人逐项确认 / 修改 / 删除 / 新增
- 拖拽调整 rect 坐标（基于 SVG viewport）
- 导出最终 `*.json`（合并 auto + manual）

5.2 实现选项

方案	优点	缺点
CLI 交互（类似 annotate_hotzone.py）	与现有流水线一致	UX 差，调坐标痛苦
Web 标注页（独立 Vite 工具）	可视化拖拽	额外维护一个工具
Admin 后台集成（推荐 ⭐）	复用 Admin 登录/权限/上传	需要后端 API 支持

推荐方案：在 Admin 端 `views/assets/floor-structures/` 下增加标注页，调用以下 API：

端点	方法	说明
`/api/floors/:floorId/structures/candidates`	GET	拉取自动抽取的候选项
`/api/floors/:floorId/structures`	PUT	提交人工审核后的最终 `floor_map.json`
`/api/floors/:floorId/render-mode`	PATCH	切换 `vector` ↔ `semantic`

API 字段契约必须先在 [API_CONTRACT_v1.7.md]() 增补，**严禁绕过协议规范**。

---

6. 前端实现规范

6.1 Flutter 端

```
// flutter_app/lib/features/assets/presentation/widgets/floor_plan_view.dart

class FloorPlanView extends StatelessWidget {
  final FloorMap floorMap;

  @override
  Widget build(BuildContext context) {
    return switch (floorMap.renderMode) {
      RenderMode.vector   => FloorPlanVectorView(floorMap: floorMap),    // 现有 flutter_svg 方案
      RenderMode.semantic => FloorPlanSemanticView(floorMap: floorMap),  // 新增 CustomPaint 方案
    };
  }
}
```

`FloorPlanSemanticView` 用 `CustomPainter` 绘制 outline / structures / windows / north，单元层用 `GestureDetector` 包裹的 `Path`（来自 `units[].polygon`）做点击。颜色全部从 `Theme.of(context).colorScheme` 读取，禁止硬编码。

6.2 Admin 端

新增组件 [admin/src/components/FloorPlanSemantic.vue]()，输入 `floor_map.json`，循环渲染 SVG。

6.3 uni-app 端

同 Admin，组件路径 [app/src/components/FloorPlanSemantic.vue]()。颜色严格走 CSS 变量（参考 [.github/instructions/uniapp.instructions.md]()）。

6.4 状态色规范（三端共用）

严格遵循 [copilot-instructions.md "UI 色彩规范"]() 中的状态色映射表：

单元状态	填充色（来源）
`leased`	`colorScheme.primary`

`vacant`	`colorScheme.error`
`expiring_soon`	`colorScheme.tertiary`
`renovating`	extension `info` + 斜纹 pattern
`non_leasable`	`colorScheme.outline`

---

7. 实施路线图

> 不写时间估计，按依赖顺序列阶段。

阶段	交付物	依赖
P0 数据契约	在 API_CONTRACT_v1.7.md 增补 `floor_map.json` v2 schema 与三个新端点	无
P1 抽取脚本	`scripts/floor_map/*.py` 五个模块 + `split_dxf_by_floor.py` 集成	P0
P2 候选数据	A 座 24 层 `*.candidates.json` 全量生成	P1
P3 标注工具	Admin 端 `floor-structures` 页面 + 三个 API	P0 + P2
P4 前端渲染	Flutter / Admin / uni-app 三端 `FloorPlanSemanticView`	P0
P5 灰度切换	按楼层粒度切 `render_mode`，对比矢量与语义两种效果	P3 + P4

---

8. 验收标准

8.1 后端

- [ ] `scripts/split_dxf_by_floor.py` 输出 v2 schema `floor_map.json`，向下兼容（`render_mode="vector"` 时旧前端仍正常）
- [ ] A 座 24 层候选项准确率：core ≥ 95%，elevator ≥ 90%，stair ≥ 90%，column ≥ 85%
- [ ] 单元测试覆盖每个 detector 模块（pytest，≥ 1 个 fixture DXF）

8.2 前端

- [ ] Flutter `FloorPlanSemanticView` 与 FloorPlan.tsx 视觉一致度 ≥ 90%（pixel diff）
- [ ] 切换 `render_mode` 不丢失单元热区状态
- [ ] 三端共用同一份 `floor_map.json`，结果像素级一致（除主题色差异）

8.3 数据

- [ ] 一份 `floor_map.json` 通过 schema 校验（建议 JSON Schema 文件落到 docs/backend/schemas/floor_map.v2.schema.json）

9. 反模式与红线

🚫 禁止在 Service / Controller / Frontend 任何位置硬编码楼层结构（核心筒位置、电梯数量等），全部从 `floor_map.json` 读取。

🚫 禁止让 `structure_detector` 跨楼层共享缓存（每层独立抽取，避免不同楼层图层污染）。

🚫 禁止直接在 DXF 上修改后保存（仅做内存层面的 layer 隐藏 / 标记色重映射，文件不可变）。

🚫 禁止前端在渲染时做坐标计算（DXF→viewport 映射必须在后端完成，前端只 consume 像素坐标）。

🚫 禁止把候选项 `source="auto"` 直接写入数据库；必须经过人工标注工具确认才能 `PUT /api/floors/:floorId/structures` 。

10. 与现有方案的兼容性

现有资产	本方案影响
SVG_HOTZONE_SPEC.md	不变，作为 `render_mode="vector"` 规范继续生效
HOTZONE_ANNOTATE_PLAN.md	不变，单元 polygon 标注流水线复用
`scripts/split_dxf_by_floor.py`	新增字段、保持向下兼容（ `schema_version="1.0"` 输出仍可被旧前端读）
数据库 `floors.svg_version` 字段	不变；新增 `floors.render_mode VARCHAR(16) DEFAULT 'vector'` 列（migration 必须遵循 database-migration.instructions.md）

11. 后续扩展（非本期）

- 自动识别"户型分析图"残留 PROXY 实体并转化为 `units[].subdivisions[]` （嵌套子单元）
- 多版本 `floor_map.json` diff 工具（用于 CAD 改造后比对结构变化）
- 机器学习辅助识别（CNN 分类 INSERT 块的语义类型，提升 auto 准确率）